

A NOVENA TO ST. JUDE, OR FOUR EDIFYING CASE STUDIES IN MATHEMATICAL PROGRAMMING

ROBERT E. D. WOOLSEY

Institute for Operations Research
Colorado School of Mines

R. E. D. WOOLSEY is principal scientist in The Institute for Operations Research, and professor at the Colorado School of Mines. He is co-owner and operator of the Woolsey Ranch in Williamson County, Texas, and a consultant to Sandia Corporation and the Information Systems Division of General Electric Company in the general areas of operations research and mathematical/integer/geometric programming. He has previously held positions with Control Data Corporation and Sandia Corporation, where he authored several major research reports. His basic interest is the study of methods to solve real world problems.

In this author's opinion, the two most basic reasons for failure in the field of mathematical programming have nothing to do with modeling ability or technical expertise. The two reasons are: (1) Failure because the modeler does not realize that the method (mathematical programming) is a *means* to an end (the problem) rather than an end in itself, and (2) Failure because a wrong answer is extrapolated from what appears to be the right method. The case studies here have been selected because they seem to exemplify these two reasons rather well. Two were problems in which this author was personally involved; therefore the models have been altered somewhat to preserve the innocent (and the guilty) from the sneers of their fellow workers (who would, of course, *never* do anything like what is presented here). The other two case studies have been repeated to this author in various bars at various O.R. conventions for some years. The only thing the author has contributed in these cases is the considerable time it took to find out what really happened.

Before embarking on the case studies, the reader must be cautioned against the well-nigh inevitable feeling of superiority that these studies provoke. We are accustomed to read in the literature of our profession only about the successes, seldom about breaking even, and never about really hitting the wall. This author has attempted to redress this problem by a prior paper¹ in which four case studies limited to integer programming were discussed. St. Jude is again invoked because, as Patron Saint of Lost Causes, it often seems to this author that our profession's inability to see its flaws clearly requires divine aid and comfort.

If this author has learned one thing from years of teaching, it is that students learn little from the success of others, because they are so sure that, given the proper circumstances, that they could have done the same thing.

¹R. E. D. Woolsey, "A Candle To St. Jude, or Four Real World Applications of Integer Programming", *INTERFACES*, Vol. 2, No. 2 (Feb. 1972), pp. 20-27.

If anything we must remember that a real pro makes the difficult look easy. It seems then that the conscientious teacher *must* include cases where he and others really fell on their faces. If the students are appropriately impressed, perhaps they will be turned loose on American Industry less likely to subtract from, than to add to, the quarterly earnings of the company so fortunate as to hire them.

The Inflexible Steel, Inc. Blast Furnace Model

I was invited to give a talk to the Amalgamated Iron and Steel Engineers in a nameless town. When I came, the Industrial Engineers group gave me the usual tour, showing me the blast furnace, the rolling mill and so forth. They then asked if they might give me their presentation on how they were using LP in blast furnace burdening. They had formed an objective function where they were minimizing the cost per ton of finished steel per ton of ore of type j times the amount of the j th ore. In symbols we have

$$\sum_{j=1}^{j=n} c_j x_j, \quad (1)$$

where c_j is the cost per ton of finished steel per ton of ore and x_j is the amount of ore of type j in tons. The first constraint was their sintering plant capacity constraint. We have that

$$\sum_{j=1}^{j=n} s_j x_j \leq S, \quad (2)$$

where s_j is tons of sinter per ton of input ore, and S is the sintering plant capacity. At this point he added that they're only dealing with seven ores. Now, so far we're minimizing, subject to one less than or equal to constraint. So, at this point, I've got the feasible solution in my head. The next constraint was the "net tons of hot metal constraint." Where h_j is the net tons of hot metal per ton of input ore, or

$$\sum_{j=1}^{j=7} h_j x_j = NTHM. \quad (3)$$

And he said this is also called our "goal constraint." They wanted to hit this right on the button, thus the equality. At this point, I think to myself: "six variables." And then he told me that they got a trainload of x_3 , the Woody Creek ore every week, and they *had* to use it up; therefore, x_3 is always a constant known beforehand. In short,

$$x_3 = k_3. \quad (4)$$

I'm thinking: "five variables." Then he stated that of the last three ores, management kept telling them that they were going to have all three, but that they had never had more than one at a time. The one they usually had was x_5 , the U. S. Fines Stock Material. And further, x_4 , the Sunrise Fine ore, they had never had any. In symbols,

$$x_4 = 0, \quad x_6 = 0, \quad x_7 = 0. \quad (5)$$

And I'm thinking: "this has got to be some kind of a joke, there are only two variables left." The last constraint turned out to be that they required,

when they used the first two ores, that they always had to be in a set ratio to each other. In symbols,

$$rx_1 = x_2. \quad (6)$$

I said, "Is that all?" He said, "Yes." I said, "You've been running this on an LP, right?" "Right." "How long have you been doing this?" "Three years." "Do you do this on your own computer?" "No, no. We buy time." "You set it up every time, is that right?" "Oh, yes." I then asked them to go through it again. I wanted to make sure I was getting it right. So, they went through it again, and it was revealed that the last constraint had three cases, the ratio case was case 3. Case 1 was where they got a trainload of x_1 and had to use it up. And case 2, ditto for the second ore. In symbols,

$$x_1 = k_1 \quad (\text{Case 1}) \quad \text{or} \quad x_2 = k_2 \quad (\text{Case 2}). \quad (7)$$

I asked them if they had any data for this model that I could see. So they brought me a listing of their last run (a case 3 situation). I looked at the data and asked for a slide rule, which was brought. I then sent them out to get me a cup of coffee. When they came back I said, "Well, the way I look at this data, you're going to have about 219 tons of Sunrise Ore, 352 tons of Comstock, 51 tons of Woody Creek (a constant), and 146 tons of U. S. Fines Stock Material." The guy said, "You know, I've never seen anybody simplex in their head before, you must have done a lot of LP's."

I then pointed out that in all three cases their model boiled down to minimizing one variable subject to that one variable being greater than or equal to a constant. In symbols,

$$\text{Minimize } x_j \quad (\text{where } j = 2, \text{ for case 1, and } j = 1, \text{ for case 2 and 3}) \quad (8)$$

$$\text{Subject to } x_j \geq \text{Constant}.$$

Further, whichever one of the last three ores is present can always be determined by the following formula:

$$x_k = \text{Constant}_1 - \text{Constant}_2 * x_j \quad (\text{where } k = 5, 6, 7). \quad (9)$$

I then pointed out that a nomogram could be easily constructed, cast in steel and bolted to the wall of the blast furnace for the operator to use. The facts of the matter turned out to be that they had been preparing data for this program, for some three years and running it on a time-sharing computer in another city.

Now I am sure that there is a self-satisfied smirk on the face of most of my readers at this point, because you are sure that *you* would never do anything like *this*. But now let's back up and ask ourselves the question that I ask my classes when I tell them this story. "Were these people bad engineers?" The answer is that they were very *good* engineers. Their model did what it was supposed to do, it even did it *optimally*. The only problem was that they got so wrapped up in the method that, after the initial formulation, the model was no longer necessary. They did it *right*, they just did it with more power than they needed. In short, they did with more when they could have done with less.

The Celebrated Brand X Washing Machine Shipping Catastrophe

In the early days of integer programming, people noticed that integer programs looked just like linear programs, with integer requirements on the

variables. Demigods of the profession showed that you could solve various algorithms in a finite number of steps. Of course, the fact that a company that tried to solve one might be bankrupt before the finite number of steps was reached was totally immaterial to most of the people at the ORSA conferences.

There was once a manufacturer of washers and dryers that had just bought a shiny new O. R. group. The O. R. group was, in fact, purchased en bloc from another company. In the early days, O. R. groups were always bought en bloc, because the guy they left behind might be the one who knew what he was doing. The O. R. group was brought in and told: "Do some of that good stuff." The O. R. group decided to do a study on shipping the washers and dryers. The first thing the study discovered was that if you buy a boxcar to go from point A to point B, you buy it empty or full. The railroad is monstrously indifferent as to whether it has anything in it. They will, on or about the date it is supposed to move, come and get it. And, on or about the date it's supposed to arrive, it will, in theory, arrive there.

The first conclusion was that, clearly, shipping partial boxcar load lots was inefficient as well as costly. The fact that "inefficient and costly" is a general description of railroads somehow escaped notice. Therefore, it was clear that the only way you should ship washers and dryers was in boxcar load lots. Now, one result of this policy is that if some wholesaler out in West Broken Toe, Wyoming, says he needs six dryers, he's got to wait until he needs a boxcar full. Well, they recommended this and somebody pointed out that the wholesalers wouldn't like that. The O. R. group's reply was essentially: "The wholesalers can eat it, this is the *optimum* plan."

So they implemented the optimum plan, formulated as a large linear program, which would tell you where and when the boxcar loads ought to be shipped. That meant, if you ordered a few washers, they put them on some palettes. Those palettes that were going in the same general direction were then kept together, and when the palettes finally got to be a boxcarload, they would fill a boxcar, put it on a train, and get it to you. Well, very rapidly, as you can imagine, they started running out of warehouse space. They had all these partial palettes sitting around, waiting until the guy could finally order some more washers and dryers. They finally were renting warehouse space across town. They then had to hire a trucking firm to take the partial palettes to other warehouses and bring them back when they were full to be put on boxcars.

After this situation had gone on for about three months, the bean counter section finally began to get interested in the rising costs in warehousing. And a bean counter came in with his green eyeshade, and his sleeve garters, and pointed out that a plot of the rising warehouse costs seemed to be best approximated by e^{x^2} . The head of the O. R. group was not at all disturbed, and pointed out that such advanced models usually take a while to get settled in. There was no question that costs would go back down; he told the bean counter to have faith. And the bean counter went away and pondered these things in his heart.

Pretty soon, the O. R. analyst who was running this LP realized that what they were really doing was solving the problem as an LP and rounding the solution down to the integer boxcar load lots they wanted. He wondered to himself, just how different the integer programming solution to this problem might be. So he called a nearby data center and asked if they could run

his problem on an integer programming code. Naturally, the man at the data center asked him how badly he wanted to solve it. He further explained that as the code was a dual code, until they got the optimal solution, they didn't even have a feasible solution. It was explained that you just paid your money and they would run until either you got the optimum solution or your money ran out. The money was put down and they ran the code for two or three hours and it hadn't converged. Various tricks were tried by the data center analysts and finally the last idea they tried worked, and the code converged in about fifteen minutes. It converged to the fact that there was no lattice point in the feasible region.

Now, let me draw a picture of that. That says the feasible region was so constrained that once they had built in all the cost constraints and everything, there was *no way* to ship integral boxcarload lots within their cost constraints. But you see, they were solving the linear program and then rounding down to the nearest integer answer, not thinking that that answer might not be feasible. So the people at the data center drew short straws to see who was going to go tell Brand X. The man who drew the short straw went over to Brand X, and he had never been there before. As a result, he just gave his answer to the first executive he found. That man happened to be the company comptroller. Now as arrogant O. R. groups tend to make mortal enemies of the bean counters, the head bean counter at Brand X was no exception. The company controller took that answer, along with the latest cost figures and went in and presented them to the president. In a four month period, that model had cost the company a little better than \$367,000 in excess shipping costs. So when the head O. R. man came back from his vacation he had no desk. There was no O. R. group there at all. And there hasn't been ever since.

The Amalgamated Conglomerate (Canada) Ltd. Model

As an integer programmer, you often get phone calls from people who are trying to solve one. One that sticks in my memory is the one where a fellow from a company in Canada called and described a capital budgeting problem as follows:

He started off by saying that it had 60,000 0-1 variables, 10,000 constraints. He then asked me what I would suggest that they do with such a problem. As I couldn't resist, I suggested that he first say a novena to St. Jude. He replied that they had heard that I was often jocular about such things. What he was doing was coming up with a five-year planning model for his company.

The 60,000 variables came about in the following way: he had 1,000 projects that could be done during any one of the following 60 months. The i th project can be started in any one of the 60 months, so j runs from 1 to 60. We have a thousand constraints looking like

$$\sum_{j=1}^{60} x_{ij} = 1, \quad i = 1, 2, \dots, 1000.$$

And 60,000 0-1 variables. His model is actually maximizing return subject to some discounted cash flow constraints. For each year he has a constraint on cash flow allowable in that year. Also, he had constraints on cash flow allowed for the first two years, the first three years, the first four years, and all five years. I then pointed out that I understood where he might get the first

year's cash flow constraint, as he was probably operating from a budget decided last year. But I did not see, at once, how he got the second year's cash flow constraint. He told me that they had extrapolated the second year's right-hand side from the value they had for the first year. I then inquired if they had extrapolated the third year from the second year the same way, and then continued the process out to the fifth year. He said that they had done so. I then pointed out that if he took a tight first year cash flow constraint and then added to it a loose second year constraint, that the result would be virtually meaningless. As he didn't understand, I tried an analogy: "If I measure the left-hand side of my trousers very carefully, and I measure the right-hand side of my trousers to the nearest foot, my pants would fall down." He said that he understood. Then I said, "Therefore, the fifth year really depends on the fourth year, which depends on the third year, which depends on the second year, which depends on the first year, which is where it ends. We now only have one year to solve so we have only 12,000 variables."

"Well," he said, "not exactly, because we got the twelfth month from the eleventh month." So we rapidly get down to one month and 1,000 variables. So, we have one month where we have this tight budget constraint that we believe in, and a thousand projects to consider. He said, "Well, we now have something we can at least look at." Then I asked, "How many hours do you work a week?" He said, "40." I said, "Everybody in the company pretty much works 40 hours?" "Sure." "160 hours a month, right?" "Right." "Well," I said, "160 hours a month divided into 1,000 project decisions implies that someone is making $6\frac{1}{4}$ decisions an hour." I asked, "Who is that?" He said, "Well, we don't do that now, but with the model we're going to do it." I said, "Now in order to calculate a discounted cash flow value or net present value, you've got to know the following things: the initial investment, the salvage value, the interest rate, the annual disbursements, and the annual income, assuming, of course, that the interest won't change from day to day. Now, let's assign each one of those things a cost of \$.50 to collect the data. But now that's today. Next week the interest rates will change and some other things will change and we'll have to recalculate. Do you think you can calculate the necessary data for 1,000 variables before they're like a carload of old fruit (i.e., rotten) and you can't use them anymore?" He thought about that. I said, "Let's consider the cost at \$.50 an item to fill in this matrix and to generate this model." It took me half an hour to show him that the cost of gathering the data, punching it, card to taping it, and generally getting it ready for the model, was going to make a noticeable difference in the gross national product of Canada. He had not considered this. I asked, "Have you begun collecting the data?" He said, he had not. I said, "Well, I have a very simple rule."

IF THE COST OF COLLECTING THE DATA FOR A MODEL IS GREATER THAN THE AMOUNT YOU CAN SAVE BY SOLVING IT, YOU SHOULDN'T DO THAT.

He said, "You are telling us that we don't have an integer programming problem." I said, "I am telling you I don't think you should proceed any further with this model before you know what the data is going to cost you. Why don't you discuss this with your superiors."

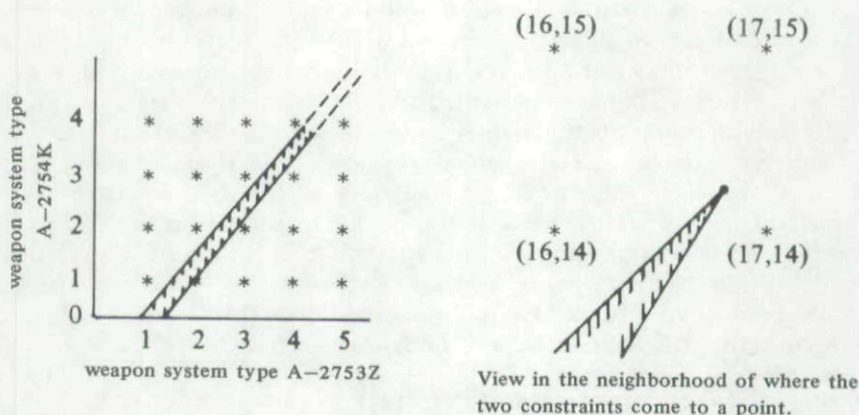
Two weeks later I got a letter, saying that he had discussed the project with his superiors and the project had been scrapped and was being re-examined in light of money available.

The Lower Slobbovian Air Battle Model Disaster

Once there was an emerging country that decided to build an air battle model that would insure optimum use of their air force against their foes. A number of foreign O. R. analysts were engaged to develop the model. They were divided into two groups, one to design the objective function to be optimized and the other to design the constraint set of the defensive network that the Slobbovian Air Force had to face.

This model was some three years in the building. And finally there came the time to run it. Everybody was brought into the machine room to see it run, the admirals and the generals and so forth. Everybody gathered around while this thing ran. They loaded it in and the tapes whirled and the printers printed, and it was wonderful. After about 20 minutes it came out and said that *all* of the planes had been shot down. The Slobbovian Air Command was not overjoyed, particularly since there was a group of admirals who had come to watch who were sitting in a corner snickering. The Slobbovian Air Force panicked. All the admirals were hosed out the door. The O. R. analysts were then told to do it again, and do it *right*.

So the model was redone. The group that had designed the objective function now got to do the constraints and the group that did the constraints now got to do the objective function. This way, they could immediately eliminate any un-Slobbovian things that could possibly have crept into the model. It was run again at two o'clock in the morning with armed guards on the door. With exactly the same result. Again, there was panic in the streets. The grunts were told to *find it*. And so, every possible combination of planes, weapons system, and sites were plotted against every other possible combination of planes and weapons systems and sites. After about two days, they found it.



Now if we look at the above illustration, we can see what happened. The linear programming code came up with an optimal value of the objective function at (16.94 of weapon system A-2753Z, and 14.03 of weapon system A-2754K). When the answer was rounded down to the point (16,14) they had generated the nearest integer number of weapons systems; unfortunately, that point was infeasible.

I have a simple minded viewpoint that says what this thing might have

been saying was that we needed to take the biggest bombers in the Slobbovian Air Force and fly them at treetop level in big, globby formations. Now one doesn't have to be too smart to figure out that if you fly bombers towards a target in the above manner that some peasant on the ground just might notice them go by and *tell* somebody.

Notice again that on the diagram above the only *feasible* point relative to these two weapon systems occurs at (2, 1). The error in this model was simply that it gave an optimum solution to a *model* of a real-world situation that was a damned bad approximation of reality. The fact that the answer it came up with showed a total lack of common sense should hardly surprise anyone. This answer could have been avoided by the simple expedient of plugging the "optimal values" of the variables back into the model, and see if all of the constraints were satisfied. Obviously, in this case, at least two constraints would have been violated by this "optimal" solution, and some adjustment could have been made. An *integer* programming problem was tried on a *linear* programming code with predictable results. Now this is *not* to say that running an integer programming problem on a linear programming code could not provide useful results. Very often, particularly in capital budgeting type problems, the l.p. values of the decision variables can be very good indicators as to which variables should be zero or one. But it should be clear to anyone that these "adjusted" answers derived from fractional values *must* be plugged back into the constraints and checked for feasibility. From the above we derive the following rule:

HE WHO CHECKS NOT HIS OPTIMUM SOLUTION AGAINST
HIS CONSTRAINTS IS INDEED DUMB.

Copyright 1973, by INFORMS, all rights reserved. Copyright of Interfaces is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.